Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Escuela de Ingeniería Informática
Universidad de Oviedo

Universidad de Oviedo
Universidá d'Uviéu
University of Oviedo

Activity 1. [Direct exchange or Bubble algorithm]

All the following tables are measured in milliseconds and WITHOUT OPTIMIZATION (-Xint) in Java. We can type “OoT” for times over one minute and “LoR” for times less than 50 milliseconds.

After measuring times, fill in the table:

N	t ordered	t reverse	t random
10000	562	2416	1581
20000	2136	9838	7031
40000	9046	40858	27465
80000	38165	OoT	OoT
160000	OoT	OoT	OoT
320000	OoT	OoT	OoT
640000	OoT	OoT	OoT
1280000	OoT	OoT	OoT

Explain whether the different times obtained agree with what is expected, according to the time complexity studied.

The Bubble Sort algorithm shows a typical quadratic time complexity, O(n^2).

Ordered Case: As N doubles (from 20,000 to 40,000), the time increases by approximately 4 times (from 2,136 ms to 9,046 ms), which is consistent with n^2 growth.

Reverse and Random Cases: These cases are significantly slower than the ordered case, quickly reaching "OoT" (Over one minute) for N being more than 80,000. This is because every element must be swapped in almost every iteration.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 2. [Selection algorithm]

Implement a SelectionTimes.java class to help you to fill in the following table:

N	t ordered	t reverse	t random
10000	449	487	444
20000	1768	1957	1771
40000	7740	9233	7267
80000	30241	32970	30214
160000	OoT	OoT	OoT
320000	OoT	OoT	OoT
640000	OoT	OoT	OoT
1280000	OoT	OoT	OoT

Explain whether the different times obtained agree with what is expected, according to the time complexity studied.

The Selection Algorithm Sort also follows a quadratic complexity, O(n^2), but with an important difference: There is consistency across all cases.

Unlike Bubble Sort, the times for ordered, reverse, and random cases are nearly identical due to the fact that the algorithm always performs the same number of comparisons to find the minimum element, regardless of the initial order of the data.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 3. [Insertion algorithm]

Implement an InsertionTimes.java class to help you to fill in the following table:

N	t ordered	t reverse	t random
10000	LoR	682	337
20000	LoR	2765	1331
40000	LoR	13005	5321
80000	LoR	OoT	23494
160000	LoR	OoT	OoT
320000	LoR	OoT	OoT
640000	LoR	OoT	OoT
1280000	LoR	OoT	OoT

Explain whether the different times obtained agree with what is expected, according to the time complexity studied.

The Insertion Algorithm drastically varies its performance depending on the initial state.

As shown above, the best case is easily when the data is ordered as it Remains "LoR" (Less than 50 ms) even for N=1,280,000$. This is because the algorithm only performs one comparison per element if the list is already sorted, resulting in linear O(n) complexity.

On the other hand, the worst case is when the array is reversed as it follows an O(n^2) complexity, reaching "OoT" much faster than the random case.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 4. [Quicksort algorithm]

Implement a QuicksortTimes.java class to help you to fill in the following table:

N	t ordered	t reverse	t random
10000	LoR	LoR	LoR
20000	LoR	LoR	LoR
40000	LoR	LoR	LoR
80000	LoR	LoR	LoR
160000	LoR	LoR	84
320000	76	83	174
640000	163	171	368
1280000	318	351	798

Explain whether the different times obtained agree with what is expected, according to the time complexity studied.

The Quicksort Algorithm is the most efficient one of this three, as it follows a O(n log n) complexity.

As you can see in the table above, it remains "LoR" for all cases up to N=80,000. Furthermore, even at the largest value (N=1,280,000), the random case only takes 798 ms.

This is significantly faster than the thousands of milliseconds required by the other algorithms for much smaller values of N.

Based on the complexities and data in the tables above, project how many days (1 day = 861,400,000 milliseconds) each of the four methods (Bubble Sort, Selection Sort, Insertion Sort, and Quick Sort) would take to sort a vector of n = 811,920,000 (that is, n = 80,000 * 24 * 26 = 80,000 * 210), in the case initially in random order?

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Bubble Sort would be extremely inefficient for this volume of data and it would take approximately 1,333 days to finish.
Even though Selection Sort would be slightly more consistent than Bubble Sort, it still suffers from its O(n^2) design in worse cases, and it would take roughly 366 days.

Despite Insertion Sort being fast for small lists, at this scale it would take approximately 285 days.

As Quick Sort is significantly the most efficient one due to its O(n log n) complexity, it would finish the entire task in only 66 seconds, that is obviously less than a day haha.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 5. [Quicksort vs Insertion algorithm for low execution times]

Implement two classes (QuicksortLowTimes.java and InsertionLowtimes.java), which after being executed serve to fill the following table:

repetitions: 50000

N	t quicksort	t insertion	t quicksort / t insertion
10	56		
15	61		
20	123		
25	138		
30	145		
35	205		
40	267		
45	292	49	5,96
50	306	55	5,56
55	311	60	5,18
60	322	66	4,88
65	355	70	5,07
70	432	75	5,76
75	504	81	6,22
80	573	87	6,59
85	594	98	6,06
90	622	104	5,98
95	653	113	5,78
100	665	112	5,94

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	2
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Explain from what value of n the algorithm of better complexity (quicksort) starts to take less time than the one of worse complexity (insertion).

Quicksort is mathematically smarter than Insertion Sort, but it is also more heavy to start up. For very small lists, Quicksort spends too much time setting up its internal work (the overhead of recursion and partitioning) while Insertion Sort finishes almost instantly.

However, Quicksort begins to demonstrate its superior complexity when the list gets large enough that Insertion Sort's quadratic growth begins to slow the program down significantly. At that point, Quicksort's efficient design finally starts to compensate its heavy startup cost.